

COMICCRAFT

DOCUMENT 9: CODE LAYOUT, READABILITY & REUSABILITY

Code Layout Checklist

S.NO	CODE QUALITY PARAMETER	DESCRIPTION	FOLLOWED	REMARKS
1	Consistent Indentation	Uniform 4-space indentation throughout all Python files	■ Yes	PEP 8 standard followed consistently
2	Proper File Structure	Logical folder organization: app/, templates/, static/, tests/	■ Yes	Clear separation of concerns
3	Meaningful Variable Names	Names reflect purpose: prompt, character_name, panel_count, art_style	■ Yes	Self-documenting variable names
4	Function / Method Names	Descriptive action names: generate_outline(), generate_story(), build_comic_layout(), save_pdf()	■ Yes	Verb-noun convention
5	Code Comments	Inline and block comments explain AI prompt logic, error handling, complex transformations	■ Yes	Strategic commenting
6	Modular Design	Separate modules for each concern: Flash, Pro, image gen, layout, export, routes	■ Yes	Highly modular architecture
7	No Redundant Code	DRY principle followed; shared utilities in json_utils.py, config.py	■ Yes	No code duplication
8	Error Handling	Graceful try/except blocks, user-friendly error messages, fallback mock mode	■ Yes	Comprehensive error coverage

Reusable Components / Modules

S.NO	COMPONENT / MODULE NAME	LANGUAGE/TECHNOLOGY	WHERE REUSED	REUSABILITY LEVEL
1	gemini_flash.py	Python (Google GenAI SDK)	Any project needing structured outline/plan generation from AI	High
2	gemini_pro.py	Python (Google GenAI SDK)	Any project needing creative story/narrative generation	High

3	image_generator.py	Python (Hugging Face Diffusers)	Any project needing AI image generation from text prompts	High
4	layout_builder.py	Python (Pillow)	Any project needing image + text assembly into layouts	Medium
5	exporters.py	Python (FPDF2)	Any project needing PDF generation with Unicode support	High
6	json_utils.py	Python (json, re)	Any project needing robust JSON extraction from LLM responses	High

Overall Code Quality Assessment

ASPECT	RATING (1-5)	COMMENTS
Code Layout & Structure	■■■■■ (5)	Excellent modular architecture with clear separation of concerns across 6 distinct modules
Readability	■■■■■ (5)	Clean naming conventions, consistent PEP 8 style, well-documented functions with docstrings
Reusability	■■■■ (4)	Components are decoupled; could be extracted as independent PyPI packages with minimal changes
Documentation / Comments	■■■■■ (5)	Comprehensive README, module-level docstrings, inline comments for complex logic
Overall Score	4.75 / 5	Production-quality codebase ready for extension and collaboration